

Simulazione e Ottimizzazione di uno Spin Glass

Autore:

Alberto Demarco

Sotto la guida di

Prof. Giuseppe Magnifico

Dipartimento Interuniversitario di Fisica "Michelangelo
Merlin"

Università degli Studi di Bari "Aldo Moro"

A.A. 2025/2026



Indice

1	Introduzione	1
2	Metodologia e Analisi del Codice	1
2.1	Gli Algoritmi di Esplorazione	1
2.2	Ottimizzazione Computazionale	2
3	Risultati e Discussione	2
3.1	Generazione della Frustrazione	2
3.2	Evoluzione di Energia e Temperatura	3
3.3	Confronto delle Energie di Rilassamento	3
3.4	Impatto del Tempo Computazionale	3
3.5	Analisi della Magnetizzazione e Reticoli	3
4	Conclusioni	5

Abstract

In questa relazione vengono presentati i risultati della simulazione di uno spin glass bidimensionale. Sono stati esplorati e confrontati diversi algoritmi di ottimizzazione stocastica, tra cui l'algoritmo Greedy, il metodo di Metropolis a temperatura fissa e il Simulated Annealing (con schedule di raffreddamento lineare ed esponenziale), per determinare la configurazione di minima energia del sistema. L'analisi dei dati evidenzia l'incapacità dei metodi puramente discesisti (Greedy) di superare le barriere energetiche tipiche dei sistemi frustrati, mostrando invece come il Simulated Annealing permetta di raggiungere energie del ground state significativamente inferiori. Viene inoltre discussa l'ottimizzazione computazionale ottenuta mediante la compilazione JIT con Numba.

1 Introduzione

I vetri di spin (spin glasses) rappresentano una classe affascinante di sistemi magnetici disordinati. A differenza dei materiali ferromagnetici o antiferromagnetici, in cui gli spin seguono pattern di ordinamento chiari e periodici, i vetri di spin sono caratterizzati da interazioni casuali e in competizione tra loro.

Il modello studiato in questo esperimento è il modello di Edwards-Anderson in due dimensioni. Il sistema è costituito da un reticolo quadrato di lato $L = 40$, in cui in ogni sito è presente uno spin di Ising $S_i \in \{+1, -1\}$. L'Hamiltoniana del sistema è definita come:

$$H = - \sum_{\langle i,j \rangle} J_{ij} S_i S_j \quad (1)$$

dove la somma è estesa ai soli primi vicini e i termini di accoppiamento J_{ij} sono estratti casualmente dall'insieme $\{+1, -1\}$.

Questa equiprobabilità introduce il fenomeno della *frustrazione*: a causa di cicli chiusi di legami contrastanti, non è fisicamente possibile soddisfare simultaneamente tutti i legami energetici del reticolo. Di conseguenza, il paesaggio energetico del sistema diviene altamente rugoso, disseminato di innumerevoli valli e minimi locali separati da barriere termodina-

miche. La ricerca dello stato fondamentale (ground state) in tali condizioni si configura come un complesso problema di ottimizzazione.

2 Metodologia e Analisi del Codice

Per studiare l'evoluzione del sistema verso configurazioni a bassa energia, sono stati implementati tre differenti algoritmi stocastici, partendo sempre da matrici di spin inizializzate casualmente.

2.1 Gli Algoritmi di Esplorazione

1. Algoritmo Greedy: Esplora il paesaggio energetico accettando esclusivamente le mosse che riducono l'energia o la mantengono costante ($\Delta E \leq 0$). Equivale a un raffreddamento istantaneo a temperatura zero ($T = 0$).

2. Metropolis a T fissa: Permette transizioni verso stati a energia maggiore ($\Delta E > 0$) con probabilità legata al fattore di Boltzmann $P = \exp(-\Delta E/T)$. È stata usata una temperatura $T = 0.5$.

3. Simulated Annealing: Inizializza il sistema a temperatura elevata ($T_0 = 5.0$) per poi raffreddarlo fino a $T_f = 0.01$. Sono state con-

frontate due leggi per i 2000 passi di simulazione:

- **Lineare:**

$$T(t) = T_0 - (T_0 - T_f) \frac{t}{t_{max}}$$
- **Esponenziale:**

$$T(t) = T_0 \left(\frac{T_f}{T_0} \right)^{t/t_{max}}$$

2.2 Ottimizzazione Computazionale

Dal punto di vista informatico, i metodi Monte Carlo richiedono milioni di aggiornamenti sequenziali. Per ovviare alla lentezza nativa di Python, si è fatto ricorso alla compilazione Just-In-Time tramite la libreria Numba.

Listing 1: Doppio ciclo ottimizzato per il calcolo dell'energia in-place.

```

1 @njit
2 def calcola_energia(reticolo,
3   J_h, J_v):
4     L = reticolo.shape[0]
5     H = 0.0
6     for i in range(L):
7         for j in range(L):
8             H += -(reticolo[i,j]
9                   * reticolo[(i
10                      +1)%L, j] * J_v
11                   + reticolo[i,j]
12                     * reticolo
13                       [(i, (j
14                          +1)%L] *
15                       J_h[i,j]
16                     ])
17
18     return H

```

Come si nota nel Listato 1, l'approccio Python-standard basato su funzioni come `np.roll` è stato scartato. Tali funzioni allocano dinamicamente nuova memoria RAM a ogni step per creare copie traslate dell'array. L'uso di cicli `for` elementari supportati da `@njit` permette di elaborare le matrici rigorosamente *in-place*, azzerando i colli di bottiglia legati alla memoria.

3 Risultati e Discussione

3.1 Generazione della Frustrazione

L'inizializzazione degli accoppiamenti ha restituito i seguenti valori:

- **Orizzontali (J_h):** 802 ferromagnetici (+1) e 798 antiferromagnetici (-1).
- **Verticali (J_v):** 796 ferromagnetici e 804 antiferromagnetici.

Questa distribuzione quasi simmetrica è la sorgente primaria della frustrazione topologica del reticolo (visibile in Figura 1 e Figura 2).

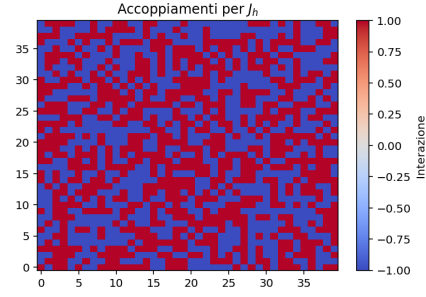


Figura 1: Mappa degli accoppiamenti orizzontali J_h .

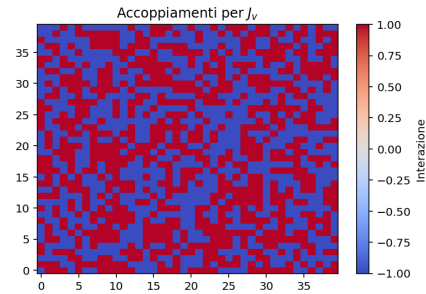


Figura 2: Mappa degli accoppiamenti verticali J_v .

3.2 Evoluzione di Energia e Temperatura

L'efficacia del Simulated Annealing è visibile analizzando la traiettoria del sistema durante i 2000 passi di ricottura (Figura 3).

Lo schedule esponenziale abbatta la temperatura in modo brusco nelle prime fasi, per poi dedicare gran parte dei passi a un lento e fine assestamento. Lo schedule lineare, invece, perde molto tempo a temperature elevate.

3.3 Confronto delle Energie di Rilassamento

I quattro approcci sono stati mediati su decine di run indipendenti. I risultati delle energie finali (E/N) sono riportati in Figura 4 e descritti di seguito:

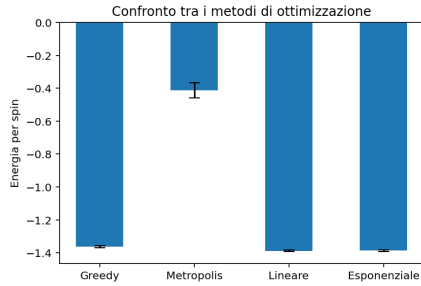


Figura 4: Confronto dell'energia media finale raggiunta.

- **Metropolis ($T=0.5$):** $E/N = -0.4117 \pm 0.0454$ (minimo assoluto -0.4675). Il valore alto indica che le violente fluttuazioni termiche impediscono la condensazione.
- **Greedy:** $E/N = -1.3630 \pm 0.0078$ (minimo -1.375). Privo di inerzia termica, l'algoritmo rimane intrappolato in un minimo locale superficiale.

- **Annealing Esponenziale:** $E/N = -1.3870 \pm 0.0047$ (minimo -1.3925).
- **Annealing Lineare:** $E/N = -1.3880 \pm 0.0060$ (minimo -1.4025).

I metodi di ricottura simulata superano nettamente l'approccio Greedy. In questa specifica serie di esecuzioni, la schedule lineare ha raggiunto un'energia media marginalmente più bassa (-1.388 contro -1.387), dimostrando eccellenti capacità di scalcare le barriere.

3.4 Impatto del Tempo Computazionale

Come mostrato in Figura 5, all'aumentare dei passi concessi alla simulazione, l'energia finale decresce sistematicamente. Dare più "tempo termico" al sistema è cruciale per fargli trovare i minimi globali.

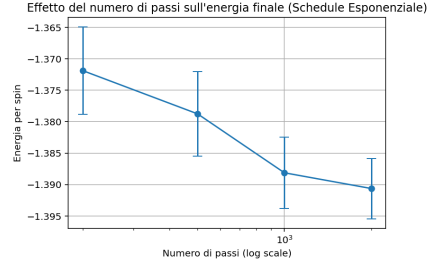


Figura 5: Dipendenza dell'energia finale dal numero totale di passi.

3.5 Analisi della Magnetizzazione e Reticoli

Per determinare l'assenza di ordini magnetici macroscopici, è stata valutata la magnetizzazione media per spin $|M|/N$. I risultati ottenuti sono: Greedy 0.0475, Metropolis 0.0112, Annealing Esponenziale 0.0262. In ogni scenario, la magnetizzazione è trascurabile. Come visibile nelle "fotografie" finali (Figura 6), non vi è alcun dominio ordinato a lungo raggio.

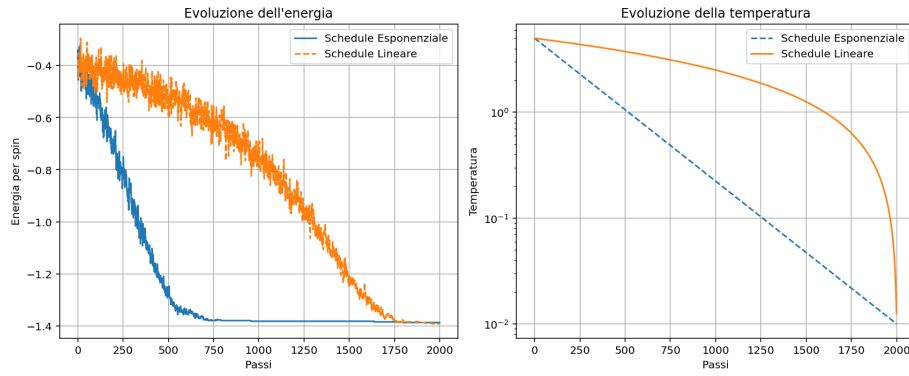


Figura 3: Evoluzione dell'energia per spin (sinistra) e della temperatura (destra) in funzione dei passi per le due schedule di raffreddamento.

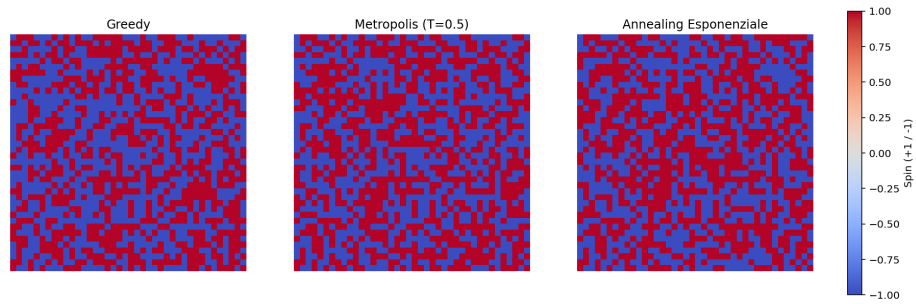


Figura 6: Confronto visivo delle configurazioni finali del reticolo per i vari metodi. Si nota la natura intrinsecamente disordinata dello spin glass.

4 Conclusioni

L'indagine ha dimostrato la forte dipendenza dello stato finale dalla dinamica di ottimizzazione. Nei paesaggi frustrati, algoritmi discesisti (Greedy) falliscono sistematicamente. Reintrodurre la temperatura abbassandola gradualmente (Simulated Annealing) fornisce l'energia necessaria per sfuggire ai minimi locali. L'uso del compilatore JIT Numba si è rivelato una scelta architetturale imprescindibile per la viabilità del calcolo statistico.